

ACARA PRAKTIKUM 8

JARINGAN SYARAF TIRUAN 3

TUJUAN

- Memahami konsep dan cara kerja *Convolutional Neural Network* (CNN)
- Mampu membangun, melatih, dan mengevaluasi model CNN untuk klasifikasi gambar.

DASAR TEORI

1. Pengantar *Convolutional Neural Network* (CNN)

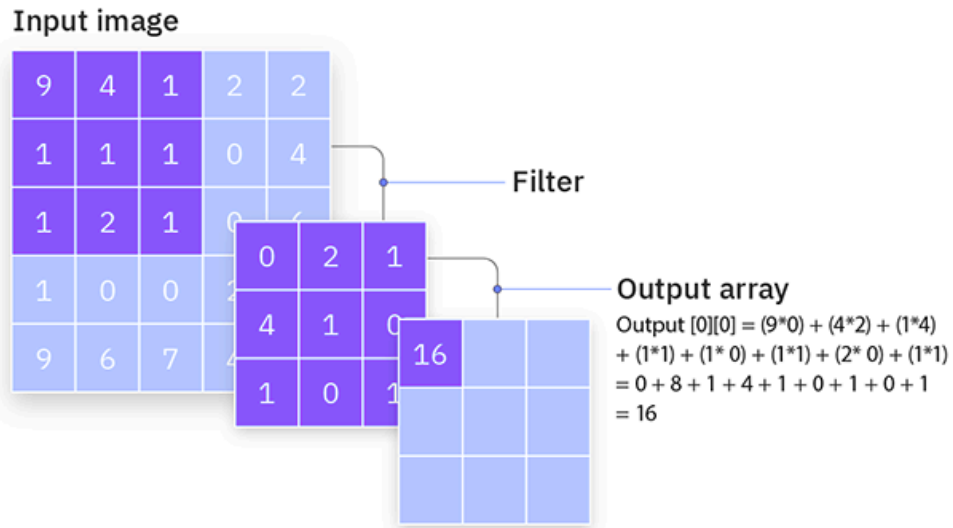
Jaringan saraf tradisional memiliki keterbatasan dalam menangani data berdimensi tinggi karena memerlukan jumlah parameter yang besar, yang dapat menyebabkan kesulitan pelatihan dan *overfitting*. Selain itu, model ini tidak mampu menangkap hubungan lokal dalam data, membuatnya kurang efektif untuk tugas yang membutuhkan pemahaman spasial atau struktur. *Convolutional Neural Network* (CNN) dikembangkan untuk mengatasi keterbatasan tersebut dengan menggunakan lapisan konvolusi yang dapat mengekstrak fitur secara efisien. CNN lebih mampu menangkap pola dan hubungan dalam data, menjadikannya efektif untuk berbagai tugas yang melibatkan data berstruktur seperti gambar atau video.

2. Arsitektur CNN

CNN (*Convolutional Neural Network*) adalah jenis jaringan saraf tiruan yang banyak digunakan untuk pengenalan citra dan pengolahan gambar. CNN terdiri dari beberapa lapisan yang memiliki peran berbeda, yaitu *Convolutional Layer*, *Pooling Layer*, dan *Fully Connected Layer*.

a. *Convolution Layer* (Lapisan Konvolusional)

Lapisan konvolusional bertanggung jawab untuk mengekstraksi fitur dari gambar input dengan menerapkan filter (kernel) ke gambar. Filter ini akan melintasi setiap piksel gambar dan menghitung *dot product* antara nilai piksel dalam gambar dengan bobot filter.



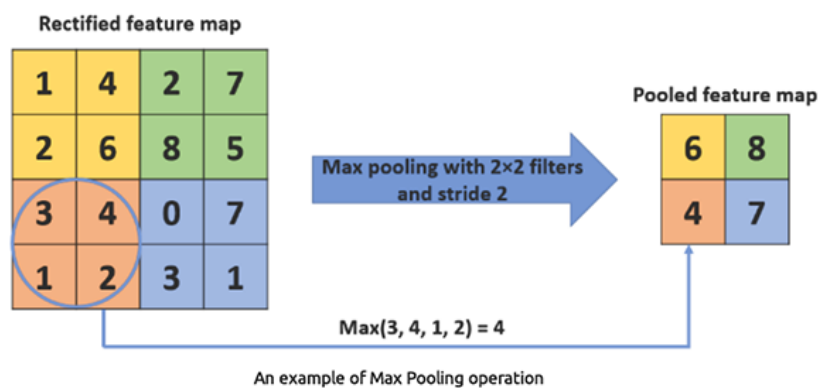
```
tf.keras.layers.Conv2D(
    filters,
    kernel_size,
    strides,
    padding,
    activation
)
```

b. *Pooling Layer*

Lapisan *pooling* bertugas untuk mengurangi dimensi spasial dari *feature map* yang dihasilkan oleh lapisan konvolusi, namun tetap menjaga fitur-fitur penting. Biasanya digunakan pooling maksimum (max pooling), yang hanya mengambil nilai terbesar dari sekumpulan nilai dalam filter.

- *Max Pooling*

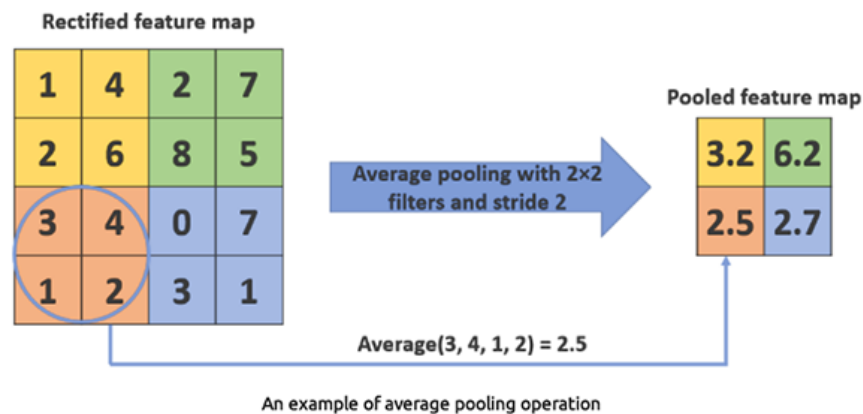
Max pooling mengambil nilai terbesar dari sekumpulan nilai dalam filter.



```
tf.keras.layers.MaxPooling2D(
    pool_size,
    strides,
    padding
)
```

- *Average Pooling*

Average pooling mengambil nilai rata-rata dari sekumpulan nilai dalam filter.



```
tf.keras.layers.AveragePooling2D(
    pool_size,
    strides,
    padding
)
```

c. *Fully Connected Layer*

Setelah beberapa lapisan konvolusi dan *pooling*, *feature map* yang dihasilkan di-*flatten* (diratakan) menjadi satu vektor. Vektor ini kemudian diproses oleh lapisan *fully connected* yang berfungsi untuk melakukan klasifikasi atau prediksi akhir.

```
tf.keras.layers.Dense(
    units,
    activation,
)
```

Keterangan:

- 1) *filters*: Matriks persegi kecil yang bergerak di atas data input untuk mendeteksi pola lokal seperti tepi, tekstur, atau bentuk tertentu. *Filters* bergerak melintasi

input data dengan langkah tertentu (*stride*). Pada setiap posisi, dilakukan perkalian antara nilai piksel input dan nilai pada filter, kemudian hasilnya dijumlahkan untuk menghasilkan nilai pada *feature map*. Contoh: 32, 64, 128.

- 2) *kernel_size/pool_size*: Dimensi ukuran filter/pooling. Contoh: 3, (3, 3), (5, 5).
- 3) *stride*: Langkah pergeseran filter saat melintasi data input. *Stride* menentukan seberapa jauh filter bergerak setiap kali melakukan konvolusi.
- 4) *padding*: Penambahan piksel ekstra (biasanya nol) di sekitar tepi data input. Ini membantu mengontrol dimensi spasial dari feature map yang dihasilkan. Contoh: *valid* (tanpa *padding*), *same* (dengan *padding*).
- 5) *activation*: Fungsi aktivasi yang diterapkan setelah operasi konvolusi untuk menambahkan non-linearitas ke dalam model, memungkinkan jaringan belajar pola yang lebih kompleks. Contoh: *relu*, *tanh*, *sigmoid*, *softmax*, *none*
- 6) *units*: Dimensi ukuran *output* prediksi atau klasifikasi. Contoh: 2, 10, 30.

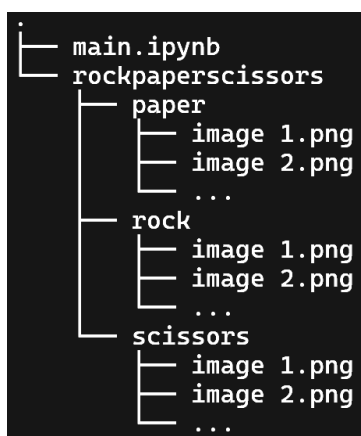
ALAT DAN BAHAN

1. Text editor/Google Colab/Kaggle Notebook/Jupyter Notebook.
2. Dataset *Rock Paper Scissor Images* yang bisa diunduh dari link berikut <https://www.kaggle.com/drgfreeman/rockpaperscissors/download>.

PERCOBAAN

Langkah-langkah:

1. Unduh dataset *rock paper scissor* dari link yang telah disediakan.
2. Ekstrak sehingga struktur foldernya menjadi



3. Import *library* yang diperlukan.

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

4. Langkah selanjutnya, akan diterapkan ImageDataGenerator untuk data latih dan data validasi. ImageDataGenerator merupakan sebuah fungsi yang sangat berguna untuk mempersiapkan data latih dan data validasi. Beberapa kemudahan yang disediakan ImageDataGenerator antara lain, preprocessing data, pelabelan sampel otomatis, dan augmentasi gambar.

```
dataset_path = "./rockpaperscissors"

train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_generator = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(150, 150),
    batch_size=32,
    class_mode='categorical',
    subset='training',
)

validation_generator = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(150, 150),
    batch_size=32,
    class_mode='categorical',
    subset='validation',
)
```

5. Setelah data siap, kita membangun model CNN yang mirip dengan MLP namun dengan penambahan layer konvolusi dan max pooling. Layer konvolusi mengekstraksi fitur penting dari gambar, sementara max pooling mengurangi resolusi gambar untuk mempercepat proses pelatihan. CNN lebih optimal dibanding MLP karena fokus pada fitur unik yang membedakan tiap kategori, bukan keseluruhan piksel gambar.

```
model = Sequential([
    Conv2D(32, (3,3), activation= 'relu', input_shape = (150,150,3)),
```

```

MaxPooling2D(2,2),
Conv2D(64, (3,3), activation= 'relu'),
MaxPooling2D(2,2),
Conv2D(128, (3,3), activation= 'relu'),
MaxPooling2D(2,2),
Flatten(),
Dense(512,activation= 'relu'),
Dense(3, activation= 'softmax')
])

```

Setelah membuat model, kita bisa menggunakan fungsi `summary()` untuk melihat `summary` dari arsitektur model yang telah kita buat.

```
model.summary()
```

6. Setelah arsitektur model CNN selesai dibuat, tahap berikutnya adalah melakukan kompilasi model dengan fungsi `compile()`. Pada tahap ini, kita menentukan *loss function* dan *optimizer* yang akan digunakan. Untuk kasus klasifikasi multi-kelas, kita menggunakan *loss function* "categorical_crossentropy", dan *optimizer* yang digunakan adalah "Adam optimizer".

```

model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy'])

```

7. Tahap terakhir dari pembuatan model adalah **model fitting**, yaitu proses pelatihan model dengan data masukan dan label yang bersesuaian. Dalam proses ini, kita menggunakan *data latih* yang sudah disiapkan dan mengoptimalkan bobot model berdasarkan *loss function* dan *optimizer* yang telah didefinisikan sebelumnya.

Dengan menggunakan **ImageDataGenerator**, kita tidak perlu secara eksplisit memasukkan gambar dan labelnya. *ImageDataGenerator* secara otomatis memberikan label sesuai dengan direktori tempat gambar berada. Contohnya, gambar di direktori "rock" akan secara otomatis dilabeli sebagai "rock."

```

history = model.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=10
)

```

8. Setelah model selesai dilatih, kita bisa mengevaluasi performa model terhadap data validasi.

```
val_loss, val_acc = model.evaluate(validation_generator)
print(f'Validation loss: {val_loss}, Validation accuracy: {val_acc}')
```

9. Kita juga dapat mencoba memprediksi hasil model pada gambar-gambar baru dari dataset validasi.

```
predictions = model.predict(validation_generator)
print(predictions) # Output berupa probabilitas prediksi tiap kelas
```

TUGAS

Upload source code percobaan di atas ke GitHub dengan nama repositori sebagai berikut,

NIM-PraktikumKB-Pertemuan8

(Tugas diselesaikan di waktu praktikum sekarang juga)